



Academic **Events API**

Plataforma REST para Gerenciamento de Eventos Acadêmicos

Módulo 3 - Desenvolvimento em C#

Grupo 6



Equipe - Grupo 6



Juliana Ballin Lima

*Desenvolvimento, documentação, testes, relatório
técnico e revisão da apresentação*

Estrutura da Apresentação



O Problema



Nossa Solução



Arquitetura



Requisito: Autenticação JWT



Requisito: Entidades e Banco de Dados



Requisito: DTOs e Services



Requisito: Repositories e Infrastructure



Requisito: Swagger / OpenAPI e Versionamento



Dificuldades e Decisões Técnicas



Demonstração Prática



Relatório Técnico em PDF



Agradecimentos

O Problema

Contexto e Motivação

Eventos acadêmicos (palestras, workshops, minicursos, seminários) são essenciais na vida universitária, mas a organização é fragmentada:



Divulgação fragmentada - grupos de WhatsApp, cartazes e e-mails avulsos



Controle de inscrições manual e sujeito a erros



Sem centralização das reações e discussões sobre os eventos

Solução



Citar que a API resolve o problema para qualquer cliente futuro, como site, app mobile ou painel administrativo.

Nossa Solução



Academic Events API - Plataforma REST centralizada

Eventos

Criar, editar, publicar
e cancelar eventos
com datas e local



Inscrições

Controle de participantes
por evento, status
Pendente / Confirmada



Comentários

Interação pública
com moderação:
só o autor pode deletar



Reações

Curtir, Adorei,
Interessante, Vou Participar
(1 por usuário por evento)



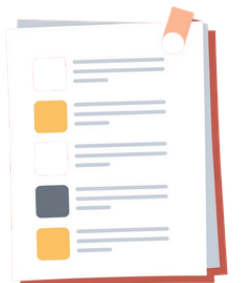
Autenticação

Cadastro e login
com JWT Bearer Token
e hash BCrypt



Documentação

Swagger/OpenAPI
interativo com botão
Authorize integrado



Arquitetura: Tecnologia

Stack Tecnológica

Camada	Tecnologia	Finalidade
Framework	ASP.NET Core 8	Criar a API REST, controllers, autenticação, autorização e pipeline HTTP
Banco de dados	PostgreSQL 15 via Docker	Persistir usuários, eventos, inscrições, comentários e reações
ORM	Entity Framework Core 8.0.11	Mapear entidades C# para tabelas, relacionamentos, índices e consultas LINQ
Driver PostgreSQL	Npgsql.EntityFrameworkCore.PostgreSQL 8.0.11	Conectar o EF Core ao PostgreSQL
Autenticação	JWT Bearer 8.0.15	Proteger endpoints com token no header Authorization: Bearer
Hash de senha	BCrypt.Net-Next 4.2.0	Salvar senhas com hash seguro e salt automático
Documentação	Swagger / Swashbuckle 6.6.2	Exibir e testar endpoints pela interface web, incluindo botão Authorize
Ambiente	Docker Compose	Subir o PostgreSQL localmente com configuração padronizada
Testes	xUnit e Moq	Testar regras de negócio dos services sem depender do banco real

Arquitetura: Em Camadas

Clean Architecture - 5 Projetos Separados

AcademicEvents.API

ASP.NET Core Web API
Controllers, configuração JWT, Swagger, DI,
Program.cs e entrada HTTP
[Ponto de entrada - referencia os outros]

entrada: controllers, JWT, Swagger

AcademicEvents.Application

Class Library C#
DTOs, interfaces de services, services, validações e
regras de negócio
[Casos de uso - depende só do Domain]

services, DTOs, interfaces

AcademicEvents.Domain

Class Library C# (puro)
Entidades, Enums, Interfaces de Repository
[Sem dependências externas]

entidades, enums, interfaces de repository

AcademicEvents.Infrastructure

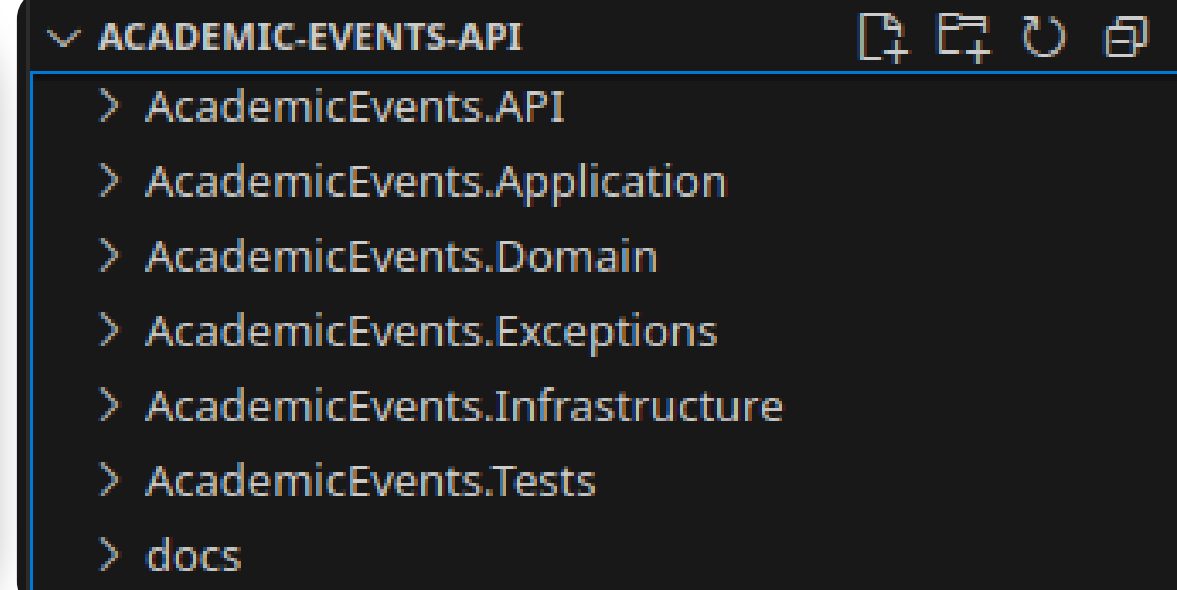
Class Library C#
DbContext, EF Core, Npgsql e repositories concretos
[Implementa interfaces do Domain]

DbContext, repositories, EF Core

AcademicEvents.Exceptions

Class Library C#
Exceções customizadas como 400, 401, 403 e 404
[Usada por API e Application]

exceções customizadas



Requisito: Autenticação - JWT Bearer Token

Endpoints públicos de autenticação:

POST /api/auth/register - cadastra usuário, retorna JWT
POST /api/auth/login - autentica, retorna JWT
GET /api/me - dados do usuário pelo token

Fluxo do Token:

1. Usuário envia { "email": "...", "senha": "..." } para POST /api/auth/login
2. AuthService valida com BCrypt
3. Token JWT gerado com claims: id, email, nome
4. Token retornado com validade de 8 horas
5. Todas as rotas protegidas exigem Authorization: Bearer {token}

Senhas nunca armazenadas em texto puro - BCrypt com salt automático.

Segurança Implementada

Hash BCrypt com salt automático (nunca texto puro no banco)

Mesma mensagem de erro para email ou senha incorretos (evita enumeração de usuários)

Token assinado com chave secreta HS256 - validação automática pelo middleware do ASP.NET

Expiração configurável no appsettings.json



Requisito: Autenticação - JWT Bearer Token

POST /api/auth/register - cadastra usuário, retorna JWT

Curl

```
curl -X 'POST' \
  'http://localhost:5000/api/auth/register' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "nome": "Maria Silva",
    "email": "maria@teste.com",
    "senha": "Senha123!"
  }'
```

Request URL

http://localhost:5000/api/auth/register

Server response

Code	Details
------	---------

200

Response body

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJodHRwOi8vc2NoZWlhcjY4bWxzbnZmLm9yZy93cy8yMDA1LzA1L2lkZW50aXR5L2NsYWltcy9uYW1laWRlbnpZmlldiI6IjEiLCJodHRwOi8vc2NoZWlhcjY4bWxzbnZmLm9yZy93cy8yMDA1LzA1L2lkZW50aXR5L2NsYWltcy9uYW1lIjoiaWFuaWEgU2lsdmEiLCJleHAiOjE3ODAzNTM5OTQsImZcyI6IjFkIiwiaXNjb2lnbmh0dG8iOiJ1b3VudHNDbGlbnRlcjY9.WZT-Y7YXreivuF7zoL_2Li-wMAQC0o6AKptIFkbj9Ek",
  "nome": "Maria Silva",
  "email": "maria@teste.com",
  "expiraEm": "2026-06-01T22:46:34.1143182Z"
}
```

Response headers

```
content-type: application/json; charset=utf-8
date: Mon, 01 Jun 2026 14:46:33 GMT
server: Kestrel
transfer-encoding: chunked
```

Responses

Code	Description
------	-------------

Links

200

OK

No links

Grupo 6

Módulo 3 - Desenvolvimento em C#

Academic Events API

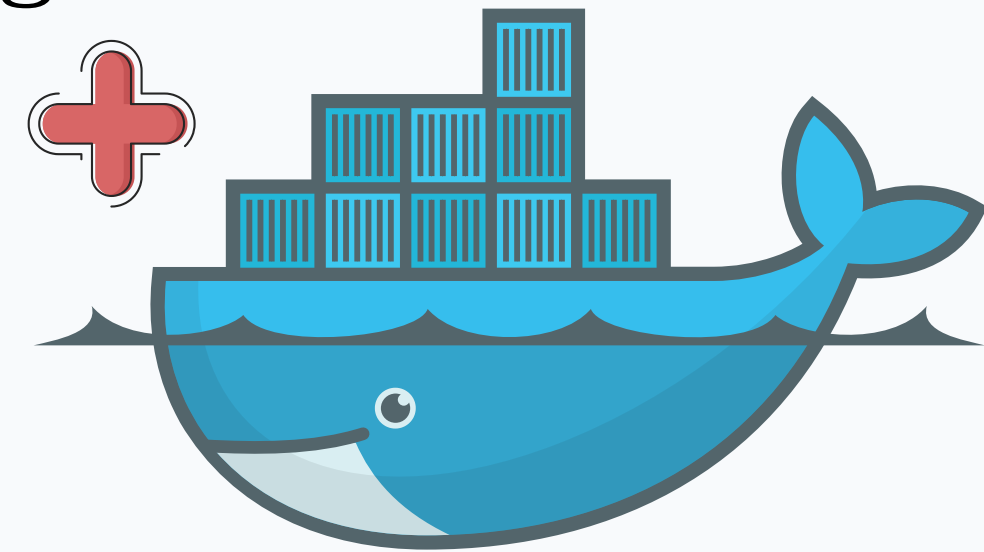
Requisito: Entidades e Banco de Dados

User	Event	Registration	Comment	Reaction
Id, Nome, Email SenhaHash, CriadoEm	Id, Titulo, Descricao DataInicio, DataFim, Local Status, OrganizadorId	Id, Usuarioid, Eventoid Status, CriadoEm (indice unico dupla-chave)	Id, Usuarioid, Eventoid Conteudo, CriadoEm	Id, Usuarioid, Eventoid Tipo, CriadoEm (indice unico dupla-chave)

Banco de Dados - PostgreSQL 15 via Docker

- EF Core 8 com Npgsql - mapeamento via Fluent API no OnModelCreating()
- EnsureCreated() no startup - cria tabelas automaticamente sem migrations manuais
- Índices únicos: User.Email, (Usuarioid, Eventoid) em Registration e Reaction
- Relacionamentos: Cascade em dependentes, Restrict para User-Event (protege organizador)
- Docker Compose na raiz - PostgreSQL 15, porta 5432, dados persistidos em volume

PostgreSQL



Requisito: DTOs e Services

DTOs (Data Transfer Objects)

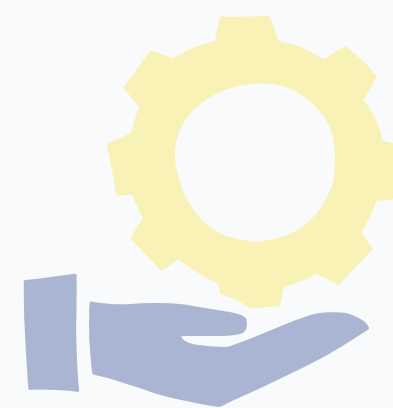
- **Request:** dados recebidos do cliente com validações DataAnnotations
- **Response:** dados retornados sem expor a entidade diretamente
- **Exemplo:** RegisterRequest valida Nome (2-100 caracteres), Email (formato), Senha (6-100 caracteres)
- UpdateEventRequest exige Status como enum - cliente envia o valor numérico

Services - Regras de Negócio

- **AuthService:** hash BCrypt, validação de credenciais, geração de JWT
- **EventService:** valida DataFim > DataInicio, verifica organizador para edit/delete
- **RegistrationService:** verifica duplicata antes do banco, lança InscricaoDuplicadaException
- **CommentService:** verifica autoria antes de deletar (UnauthorizedException)
- **ReactionService:** verifica reação existente, uma por usuário por evento

Padrões seguidos

- Controller só repassa para o service
 - Nunca tem regra de negócio
- Service só usa interfaces do Domain
 - Não conhece a Infrastructure
- Injeção de dependência via DI container
 - Cada service registrado como Scoped
- Método MapearParaResponse() privado
 - Conversão de entidade para DTO no service



Requisito: Repositories e Infrastructure

Repository Pattern

- **5 repositories:** User, Event, Registration, Comment, Reaction
- **Interfaces** definidas no Domain (IEventRepository, IUserRepository...)
- Implementações concretas na Infrastructure usando **EF Core**
- Include() nos gets para evitar N+1 queries
- **OrderByDescending**(CriadoEm) nas listas de comentários e reações

Extension Methods de DI

- **AddInfrastructure**(config) - registra DbContext e 5 repositories como Scoped
- **AddApplication**() - registra 5 services como Scoped
- **Program.cs** chama os dois, mantendo o arquivo limpo

Exceções Customizadas

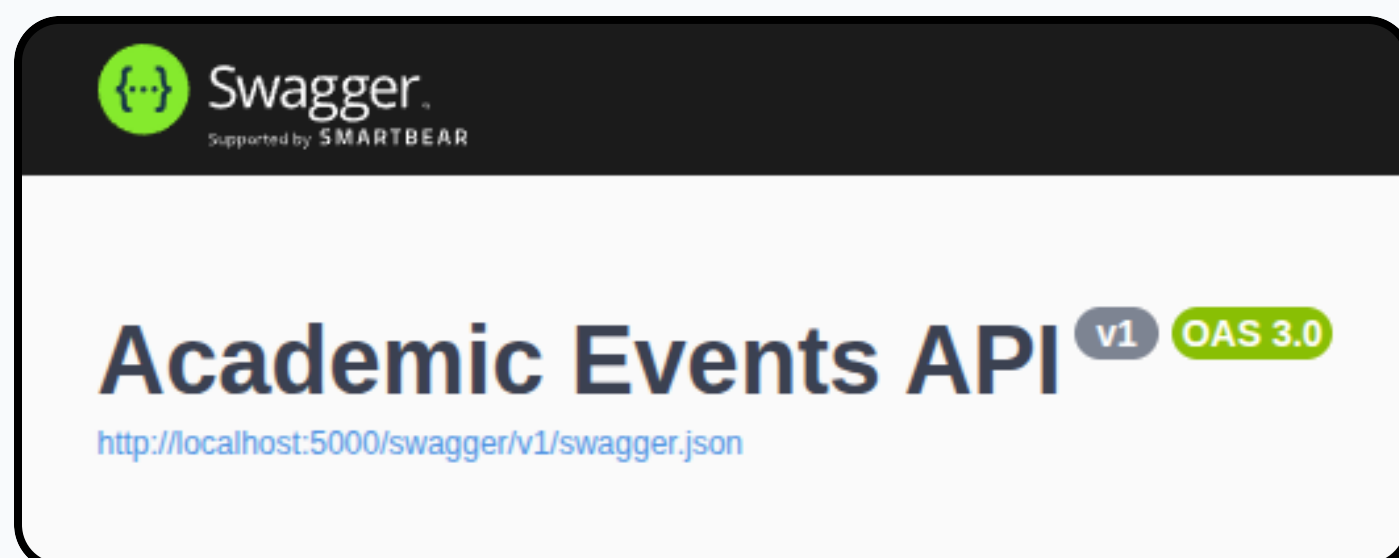
- NotFoundException (404)
 - Recurso não encontrado no banco
- DuplicateEmailException (400)
 - Email já cadastrado no sistema
- InvalidCredentialsException (401)
 - Email ou senha inválidos
- UnauthorizedException (403)
 - Usuário sem permissão para a ação
- InscricaoDuplicadaException (400)
 - Usuário já inscrito no evento



Requisito: Swagger / OpenAPI e Versionamento

Swagger / OpenAPI

- **Swashbuckle.AspNetCore 6.6.2** habilitado no Program.cs
- Botão **Authorize** integrado - suporte a Bearer Token
- Todos os endpoints documentados com ProducesResponseType
- [Produces(application/json)] em todos os controllers
- Acesso em: <http://localhost:5000/swagger>



Versionamento - Git

Branch develop para desenvolvimento, main para produção
Commits convencionais: feat(Arquivo): descrição



Endpoints completos da API

- POST /api/auth/register
- POST /api/auth/login
- GET /api/me
- GET /api/events
- GET /api/events?status=X
- GET /api/events/{id}
- GET /api/events/meus (auth)
- POST /api/events (auth)
- PUT /api/events/{id} (auth)
- DELETE /api/events/{id} (auth)
- POST /api/registrations (auth)
- GET /api/registrations/me (auth)
- DELETE /api/registrations/{id} (auth)
- GET /api/comments?eventold=X
- POST /api/comments (auth)
- DELETE /api/comments/{id} (auth)
- GET /api/reactions?eventold=X
- POST /api/reactions (auth)
- DELETE /api/reactions/{id}

Dificuldades e Decisões Técnicas

Versões NuGet

A combinação de versões do EF Core, Npgsql e ASP.NET exigiu atenção - usamos 8.0.11 para compatibilidade total com .NET 8.

Forbid() vs StatusCode(403)

Descobrimos que Forbid(mensagem) não retorna a mensagem no body - usa o parâmetro como nome de esquema de autenticação.
Solução: StatusCode(403, ex.Message).

Application sem referência à Infrastructure

A camada Application só conhece interfaces do Domain. A Infrastructure é injetada pela API via DI - isso garante inversão de dependência correta.

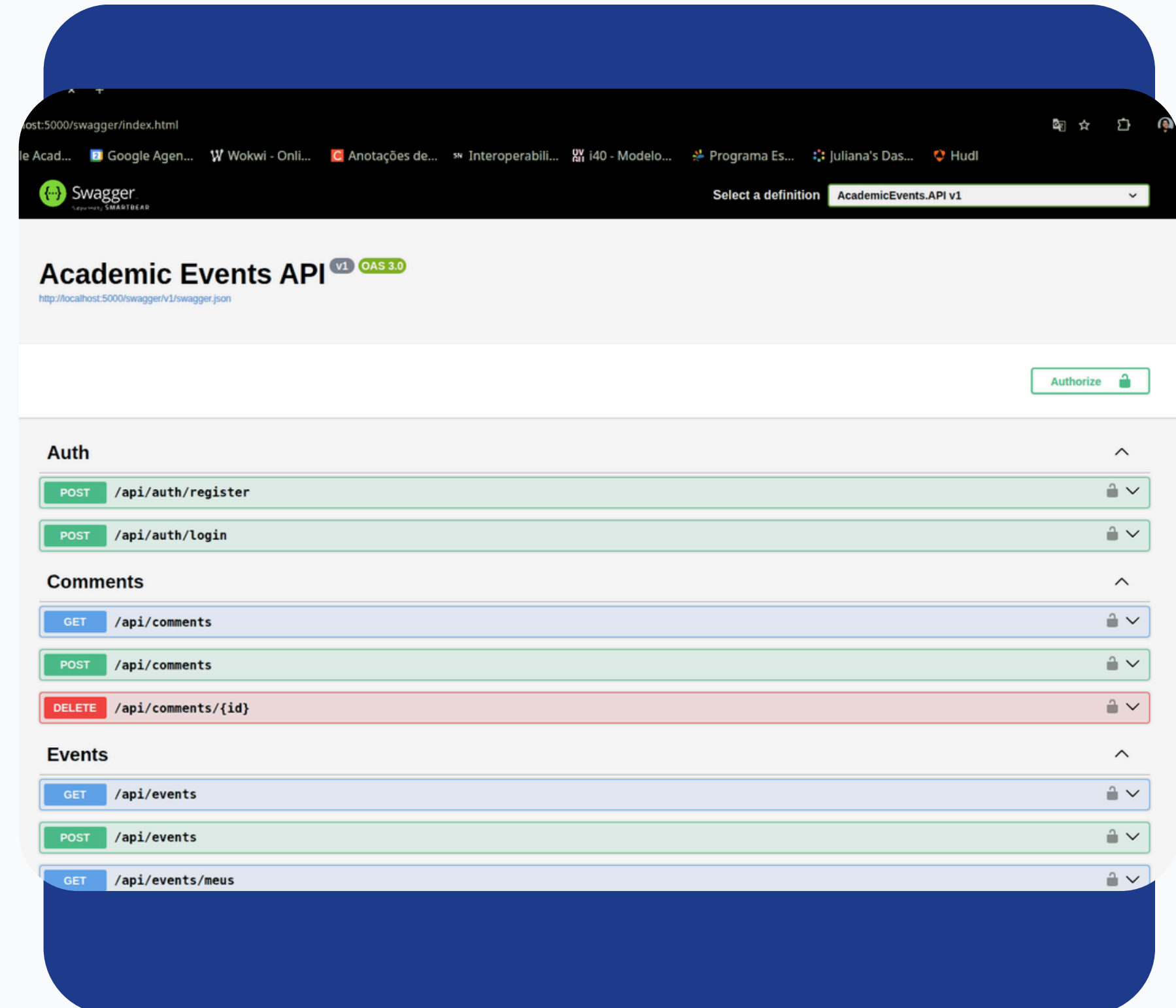
Validação em duas camadas

Para inscrições e reações duplicadas, validamos no Service antes do banco e temos índice único no banco. O service dá mensagem amigável; o banco é a última barreira.

Demonstração Prática

Roteiro da demonstração ao vivo:

- Abrir o Swagger em <http://localhost:5000/swagger>
- Cadastrar um usuário via POST `/api/auth/register`
- Fazer login e copiar o token JWT retornado
- Clicar em Authorize no Swagger e colar o token (Bearer ...)
- Criar um evento via POST `/api/events`
- Listar eventos públicos via GET `/api/events`
- Filtrar por status: GET `/api/events?status=Publicado`
- Listar meus eventos: GET `/api/events/meus`
- Fazer inscrição via POST `/api/registrations`
- Tentar inscrever de novo - observar o erro 400
- Adicionar comentário e reação
- Tentar deletar comentário de outro usuário - observar 403



Agradecimentos

Ao professor Antony Souza

Pelo projeto de referência (MiniSocial), materiais de apoio, pelas aulas e pela paciência com nossas dúvidas ao longo do módulo.

A equipe organizadora do curso

Pela organização do ambiente de aprendizado e pelo suporte contínuo.